

RAG Chatbot Project

Introduction

Ollama vlm llama2

- Goal: Set up a simple RAG (Retrieval-Augmented Generation) Chatbot.
- Key Technologies:
 - LangChain: Orchestrates components.
 - ChromaDB: Vector store.
 - Local Embedding Model: For creating numerical representations of text.
 - Local LLM (Ollama): For generating responses (e.g., `llama2`).

Project Structure

- Organize your project files like this:

```
my_rag_chatbot/  
├── data/  
│   ├── your_document.pdf  
│   └── another_document.txt  
├── chroma_db/ (Auto-created after  
ingestion)  
├── requirements.txt  
├── ingest.py  
└── chat.py
```

Step 1: Prerequisites

- Python 3.9+: Install from [python.org](https://www.python.org/).
- Git: (Optional, for version control/cloning).
- Ollama (for local LLM):**
 - Download & install from ollama.com.
- Pull an LLM model: Open terminal and run ``ollama run llama2`` (type ``bye`` to exit after download).

Step 2: Set Up Your Project

1. Create Project Directory:

- ``mkdir my_rag_chatbot``
- ``cd my_rag_chatbot``

2. Create Virtual Environment (Recommended):

- ``python -m venv venv``

3. Activate Virtual Environment:

- Windows: ``.\\venv\\Scripts\\activate``
- macOS/Linux: ``source venv/bin/activate``

Step 3: Create `requirements.txt`

- Create `requirements.txt` in
`my\_rag\_chatbot/` with:

langchain

langchain-community

chromadb

pypdf

unstructured

sentence-transformers

ollama

tiktoken

Step 4: Prepare Your Data

- Install Dependencies:
 - ``pip install -r requirements.txt``
- 1. Create ``data/`` Directory:
 - ``mkdir data``
- 2. Add Your Documents:
 - Place PDF and/or text files (e.g., ``your_document.pdf``, ``another_document.txt``) inside the ``data/`` directory.

Step 5: Create `ingest.py`

- This script processes documents for the vector database.

Step 6: Run `ingest.py`

- Execute from terminal(with virtual environment active):
 - ``python ingest.py``
- What it does:
- Loads documents from ``data/``.
- Splits them into text chunks.
- Generates embeddings using a local Sentence Transformer model.
- Saves embeddings and chunks to ``chroma_db/``.
- A ``chroma_db/`` directory will appear upon completion.

Step 7: Create `chat.py`

- This script enables interaction with the chatbot.

Step 8: Run `chat.py` and Interact

- Run the chatbot script:
 - ``python chat.py``
- Interaction:
 - Type your questions in the terminal.
 - The chatbot will retrieve information from your documents and use ``llama2`` to answer.
 - Type ``exit`` to quit.

Conclusion

- You now have a functional RAG chatbot!
- Key Components: LangChain for orchestration, local vector store (ChromaDB), local embeddings, and local LLM (Ollama).
- Further Enhancements: More data, UI integration, chat history, experimenting with different models.